



۱ پیاده‌سازی مقاله Thermal-Aware Standby-Sparing Technique on Heterogeneous Real-Time Embedded Systems

۲ مقدمه

در این مقاله، یک روش مبتنی بر Standby-Sparing دما-آگاه برای سیستم‌های نهفته و بی‌درنگ ناهمگن ارائه شده است. در این روش، ضمن در نظر گرفتن روش تحمل پذیری خطای Standby-Sparing، که در آن وظایف اصلی^۱ و پشتیبان^۲ به ترتیب بر روی دو هسته High Performance و Low Performance اجرا می‌شود. همچنین با استفاده از معیاری به نام Thermal Safe Power (TSP)، قصد دارد محدودیت‌های دمایی لازم را بر روی هسته‌ها در نظر بگیرد و در حین رعایت این محدودیت‌های دمایی، موعد زمانی وظایف بی‌درنگ نرم^۳ را رعایت کند و همچنین Quality of Service (QoS) وظایف را بیشینه کند.

۳ گزارش

در ادامه به گام‌هایی که برای پیاده‌سازی مقاله انجام شده‌اند، یکی یکی اشاره کرده و هرکدام را توضیح می‌دهم.

۱.۳ ساخت DAG مجموعه وظایف

در ابتدا مطابق با مدل سازی مسئله که مجموعه وظایف آن به صورت یک DAG است، نیاز داریم که بتوانیم DAG رندومی از مجموعه وظایف بنچمارک MiBench بسازیم. بدین منظور از مقاله

A standard task graph set for fair evaluation of multiprocessor scheduling algorithms

استفاده کردم که در آن چند روش مختلف برای ساخت Task Graph استفاده شده است. من از روش layrprob استفاده کردم. در این روش، ما تعداد لایه‌ها را مشخص کرده و پس از آن، به یک احتمال مشخص p ، بین هر یک از راس‌های لایه بالاتر با رئوس لایه‌های پایینتر آن یال برقرار می‌کنیم به طوری که هر راس از لایه‌های دوم به بعد (لایه اول که در اصل ریشه‌های درخت هستند نیازی به پدر ندارند) باید حداقل یک پدر از لایه دقیقاً بالاتر خود داشته باشند. تا شماره لایه‌ی تخصیص داده شده به آنها منطقی باشد.

^۱ Primary

^۲ Spare

^۳ Soft real-time tasks

فایل‌های کد مربوط به این بخش، در پوشه src و داخل پوشه offline_simulator/dag_creator موجود است. حال به موجودیت تسک یا وظیفه می‌پردازم:

۲.۳ کلاس Task

```

1 class Task:
2     def __init__(self, id, task_name):
3         self.id = id
4         self.task_name = task_name
5         self.parents = []
6         self.children = []
7         self.level = None
8         self.deadline = None # we will set this (I don't know how :) ) -- one value
9         self.wcet_LO = None # assigned automatically according to the task name -- per voltage/frequency value
10        self.wcet_HI = None # assigned automatically according to the task name -- per voltage/frequency value
11        self.peak_power_value_LO = None # assigned automatically according to the task name -- per voltage/frequency value
12        self.peak_power_value_HI = None # assigned automatically according to the task name -- per voltage/frequency value
13
14    def __str__(self):
15        return f"Task {self.task_name}"
16
17    def add_parent(self, parent):
18        self.parents.append(parent)
19
20    def add_child(self, child):
21        self.children.append(child)

```

شکل ۱: Task.py

همانطور که از روی تصویر مشخص است، برای هر وظیفه تعدادی پارامتر داریم که شامل موارد زیر است:

۱. deadline یا موعد زمانی مطلق
۲. parents یا پدران این وظیفه (کسانی که اجرای این وظیفه به اجرای آنها بستگی دارد)
۳. children یا فرزندان این وظیفه (وظایفی که اجرای آنها به اجرای این وظیفه بستگی دارد)
۴. wcet_hi یا بدترین زمان اجرای این وظیفه روی هسته‌های با توان مصرفی بالا یا big
۵. wcet_lo یا بدترین زمان اجرای این وظیفه روی هسته‌های با توان مصرفی پایین یا LITTLE
۶. peak_power_value_LO یا بالاترین توان مصرفی اجرای این وظیفه روی هسته با توان مصرفی پایین یا LITTLE
۷. peak_power_value_HI یا بالاترین توان مصرفی اجرای این وظیفه روی هسته با توان مصرفی بالا یا big

حال به توضیح DAG وظایف می‌پردازم.

۳.۳ کلاس DAG

در ابتدا نیاز است که تعریف درجه موازیت یا parallelism_mode را مطابق مقاله اصلی تعریف کنیم. در این مقاله گفته شده که اگر ارتفاع درخت، بین $1 \leq h < n/3$ باشد، بالاترین درجه موازیت یا high parallelism

را داریم. اگر بین $n/3 \leq h < 2n/3$ باشد درجه موازیت متوسط یا medium parallelism داریم و اگر $2n/3 \leq h < n$ باشد، کمترین درجه موازیت یا low parallelism داریم. بنابراین برای ساخت DAG ابتدا باید تعداد رئوس آن را تعیین کرده و سپس درجه موازیت را انتخاب کنیم.

سپس طبق درجه موازیت، مقدار h به طور رندوم در بازه گفته شده مشخص می‌شود. سپس رئوس با احتمال یکنواخت در این لایه‌ها تقسیم شده و سپس بین هر راس از لایه i ام و رئوس لایه پایتتر خود، با احتمال یکسان p یال برقرار می‌شود. کد کلاس DAG در زیر قرار گرفته است.

```

11 class DAG:
12     def __init__(self, n, parallelism_mode, task_set):
13         self.n = n
14         self.parallelism_mode = parallelism_mode
15         self.h = self.get_h_range_for_parallelism_mode() # assigning random height according to parallelism mo
16         self.task_set = task_set
17         self.tasks = []
18         self.levels = []
19
20     def get_h_range_for_parallelism_mode(self):
21         if self.parallelism_mode == "high":
22             start = 1
23             end = math.ceil(self.n / 3) - 1
24             return random.randint(start, end)
25
26         elif self.parallelism_mode == "medium":
27             start = math.ceil(self.n / 3)
28             end = math.ceil(2*self.n / 3) - 1
29             return random.randint(start, end)
30
31         elif self.parallelism_mode == "low":
32             start = math.ceil(2 * self.n / 3)
33             end = self.n - 1
34             return random.randint(start, end)
35
36     def get_parallelism_type(self):
37         if 1 <= self.h < self.n/3:
38             return "high"
39         elif self.n/3 <= self.h < 2*self.n/3:
40             return "medium"
41         elif 2*self.n/3 <= self.h < self.n:
42             return "low"
43         else:
44             return "INVALID_HEIGHT!"
45
46     def create_DAG(self, edge_prob=0.2):
47         if self.get_parallelism_type() == "INVALID_HEIGHT!":
48             print('Bad Height Value! DAG creation failed.')
49             return

```

شکل ۲: DAG.py

```

46 def create_DAG(self, edge_prob=0.2):
47     if self.get_parallelism_type() == "INVALID_HEIGHT!":
48         print('Bad Height Value! DAG creation failed.')
49         return
50
51     task_names = random.choices(self.task_set, k=self.n)
52
53     parts = [1] * self.h
54     remaining = self.n - self.h
55
56     for _ in range(remaining):
57         parts[random.randint(0, self.h - 1)] += 1
58
59     random.shuffle(parts)
60
61     levels = [[None] * count for count in parts]
62     self.levels = levels
63
64     task_id = 1
65     for level in range(self.h):
66         for i in range(len(self.levels[level])):
67             task = Task(task_id, task_names[task_id-1])
68             self.levels[level][i] = task
69             task.level = level
70             self.tasks.append(task)
71             task_id += 1
72
73     for i in range(self.h - 1):
74         for j in range(i + 1, self.h):
75             for parent in self.levels[i]:
76                 for child in self.levels[j]:
77                     if random.random() < edge_prob:
78                         parent.add_child(child)
79                         child.add_parent(parent)
80
81     for i in range(1, self.h):
82         for node in self.levels[i]:
83             for parent in node.parents:
84                 if random.random() < edge_prob:
85                     parent.add_child(node)
86                     node.add_parent(parent)

```

شکل ۳: DAG.py

```

46 def create_DAG(self, edge_prob=0.2):
47     self.levels = levels
48
49     task_id = 1
50     for level in range(self.h):
51         for i in range(len(self.levels[level])):
52             task = Task(task_id, task_names[task_id-1])
53             self.levels[level][i] = task
54             task.level = level
55             self.tasks.append(task)
56             task_id += 1
57
58     for i in range(self.h - 1):
59         for j in range(i + 1, self.h):
60             for parent in self.levels[i]:
61                 for child in self.levels[j]:
62                     if random.random() < edge_prob:
63                         parent.add_child(child)
64                         child.add_parent(parent)
65
66     for i in range(1, self.h):
67         for node in self.levels[i]:
68             has_direct_parent = any(parent.level == i - 1 for parent in node.parents)
69             if not has_direct_parent:
70                 parent = random.choice(self.levels[i - 1])
71                 parent.add_child(node)
72                 node.add_parent(parent)
73
74     print("DAG created successfully using layrprob method!")
75
76 def __str__(self):
77     output = ""
78     for task in self.tasks:
79         children_ids = [child.id for child in task.children]
80         parent_ids = [parent.id for parent in task.parents]
81         output += f"Task ID {task.id} ({task.task_name}) in level {task.level} -> Children: {children_ids}"
82     return output

```

شکل ۴: DAG.py

مجموعه تسک‌ها نیز از بین تسک‌های موجود در MiBench مانند basicmath، JPEG bitcount و ... انتخاب شده‌اند.

۴.۳ کد نهایی سازنده DAG

در نهایت مطابق کد زیر، نحوه فراخوانی این کد سازنده DAG، مانند زیر خواهد بود:

```

4 def run(n, parallelism_mode, show_task_names):
5     task_names = ["Basicmath", "Bitcount", "CRC32", "Dijkstra", "FFT", "Jpeg", "Patricia", "Qsort", "SHA", "String search", "S
6
7     dag = DAG(n, parallelism_mode, task_set=task_names)
8     dag.create_DAG()
9
10    print(dag)
11
12    dag.draw_dag(show_task_names)
13
14    print('-----')
15
16    print('DO YOU WANT TO SAVE THIS DAG INTO JSON FILE? (y/n)')
17
18    answer = input()
19    if answer == 'y':
20        json_data = dag.to_json()
21        with open("output/dag.json", "w") as f:
22            f.write(json_data)
23
24
25 if __name__ == "__main__":
26     if len(sys.argv) != 4:
27         print("Usage: python main.py <n> <parallelism mode including 'high', 'medium' and 'low'> <show task names y/n>")
28         print("Example: python main.py 30 high n")
29         sys.exit(1)
30
31     n = int(sys.argv[1])
32     parallelism_mode = sys.argv[2]
33     show_task_names = True if sys.argv[3] == 'y' else False
34     run(n, parallelism_mode, show_task_names)

```

شکل ۵: DAG_creator.py

```

1 python DAG_creator.py <n> <parallelism mode including 'high', '
    medium' and 'low'> <show task names y/n>

```

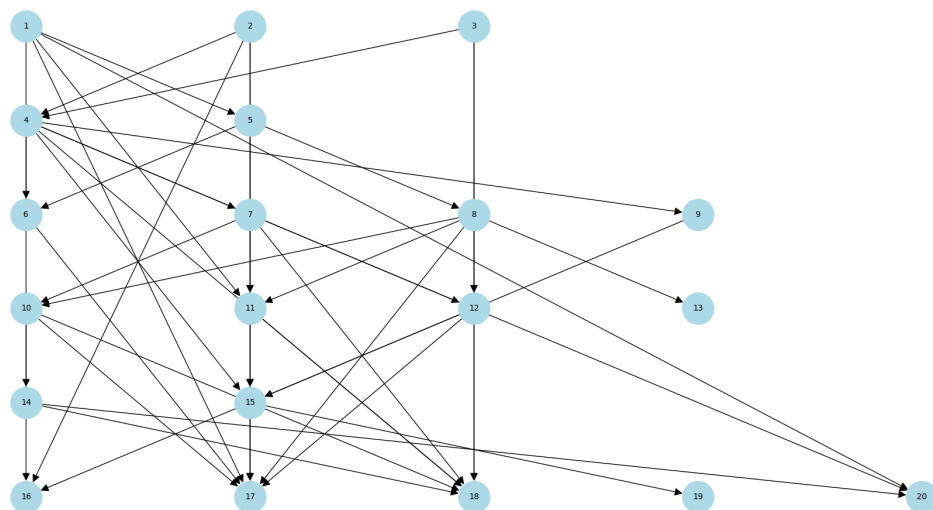
برای مثال:

```

1 python DAG_creator.py 20 high n

```

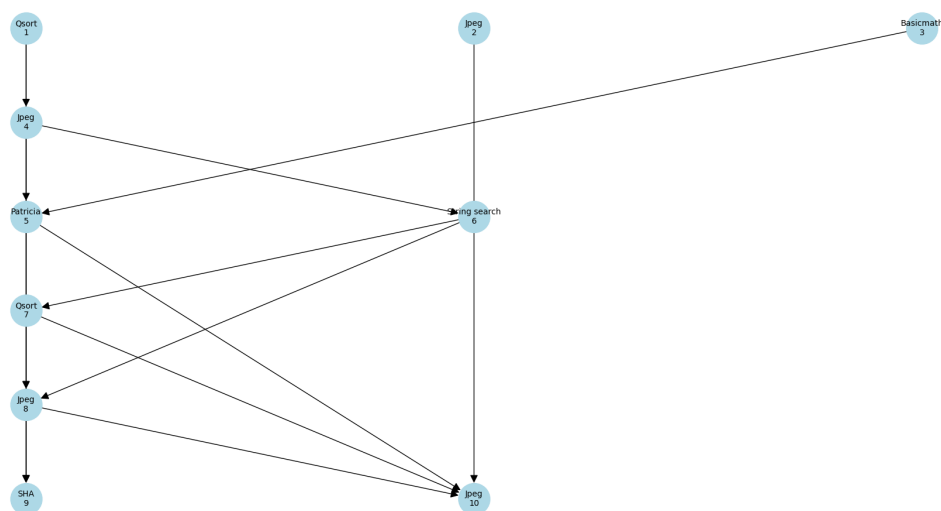
که در این مثال یعنی یک DAG وظایف با ۲۰ وظیفه که درجه موازیت آن بالا است بساز و برای نمایش، نام وظایف را در شکل نهایی نشان نده (برای شلوغ نشدن تصویر گراف). همچنین در نهایت این سوال پرسیده می‌شود که آیا می‌خواهید که این DAG در فایل جیسون ذخیره شود یا خیر که این فایل جیسون ذخیره شده می‌تواند در ادامه برای زمان‌بندی استفاده شود.



شکل ۶: نمونه DAG ساخته شده

در صورتی که دستور زیر اجرا شود، نام وظایف نیز به نمایش در خواهند آمد:

```
1 python DAG_creator.py 10 medium y
```



شکل ۷: نمونه DAG ساخته شده

۵.۳ زمان‌بند آفلاین TASS، بخش اصلی مقاله

حال وارد الگوریتم زمان‌بند آفلاین مقاله می‌شوم. شبه‌کد این الگوریتم در مقاله به شکل زیر است:

Algorithm 1. The mapping and scheduling policy of TASS

Inputs: Φ : The set of tasks in graph, TSP for all possible number of active cores, the set of core pairs $CP=\{CP_1, CP_2, \dots, CP_k\}$.
Output: The task scheduling S_i on each core C_j .

BEGIN

1. $TQ = \emptyset, PQ = \emptyset$; // Initialize the Temp. and priority queues
2. **while** $\Phi \neq \emptyset$ **do**
3. $TQ = \Phi.return()$; // Return leaves of graph
4. $T_i = TQ.select()$; // Return the task with the latest deadline in PQ
5. $PQ.put(T_i)$; // Put T_i to the priority queue (a double-ended queue)
6. $\Phi.remove(T_i)$; // Remove T_i from the graph
7. **end while**
8. **while** $PQ \neq \emptyset$ **do**
9. $T_i = PQ.select()$; // Select T_i from the priority queue PQ
10. $\varphi = minutilization\{CP_m, 1 \leq m \leq k\}$;
11. $k = Max.Finish_time_of_predecessors(T_i)$;
- **Scheduling T_i on the primary core of CP_m**
12. **while** $k \leq D_i - WC_i^{HP}$ **do**
13. $t \leftarrow$ Find first free time slot after k on $\varphi.S_{primary}$;
14. $TSP.\varphi.S_{primary} = TSP(\#active_cores \text{ at } t)$; // Return TSP of $\varphi.S_{primary}$ in HP island at t
15. **if** $Max.Power(T_i) \leq TSP.\varphi.S_{primary} \text{ at time } t$ **do**
16. Schedule T_i at t on $\varphi.S_{primary}$;
17. $PQ.remove(T_i)$;
18. **break**;
19. $k = t + 1$;
20. **end while**
21. $k =$ Finish time of T_i on $\varphi.S_{primary}$;
- **Scheduling B_i on the spare core of CP_m**
22. **while** B_i is not scheduled **do**
23. $t \leftarrow$ Find first free time slot after k on $\varphi.S_{primary}$;
24. $TSP.\varphi.S_{spare} = TSP(\#active_cores \text{ at } t)$; // Return TSP of $\varphi.S_{spare}$ in LP island at t
25. **if** $Max.Power(B_i) \leq TSP.\varphi.S_{spare}$ **do**
26. Schedule B_i at t on $\varphi.S_{spare}$;
27. **break**;
28. **end while**
29. **end while**

END

شکل ۸: TASS offline scheduling pseudo code

این الگوریتم از الگوریتم LDF یا Latest Deadline First استفاده کرده و پس از ریختن وظایف در پشته، برای زمان‌بندی آنها، تو وظیفه اصلی و پشتیبان در نظر می‌گیرد. وظیفه‌های پشتیبان باید روی هسته Spare یک جفت هسته اجرا شوند در حالی که وظیفه اصلی باید روی هسته اصلی همان جفت هسته اجرا شود. برای انتخاب جفت هسته مد نظر برای map کردن وظیفه، جفت هسته با کمترین utilization در واقع جفت هسته‌ای که کمترین زمان اجرا را تا کنون داشته‌اند، انتخاب می‌کند.

سپس بیشترین زمان پایان وظایف پدر وظیفه فعلی پیدا شده، زیرا وظیفه فعلی باید بعد از پدرانش که به آنها وابستگی دارد، اجرا شود. حال وظیفه اصلی را روی هسته اصلی این جفت هسته، پس از بیشترین زمان پایان وظایف پدر، به گونه‌ای زمان‌بندی کرده که محدودیت TSP رعایت شود (درباره این محدودیت در ادامه توضیح داده خواهد شد). همچنین برای زمان‌بندی وظیفه پشتیبان نیز، از آنجا که قصد داریم که هیچ overlap ای با وظیفه اصلی نداشته باشد، زمان پایان وظیفه اصلی را محاسبه کرده و پس از آن، وظیفه پشتیبان را به گونه‌ای زمان‌بندی می‌کنیم که TSP مجدداً رعایت شود.

حال درباره TSP توضیح می‌دهم. طبق خود مقاله TSP:

Thermal Safe Power (TSP): Efficient Power Budgeting for Heterogeneous Manycore Systems in Dark Silicon

TSP یک مقدار ایمن برای بیشینه توان مصرفی یا Peak Power یک هسته است که طبق تعداد هسته‌های فعال در پردازنده تعیین می‌شود. درباره جزئیات این محاسبه صحبت نخواهیم کرد، زیرا خود TSP یک شبیه‌ساز آماده دارد که استفاده از آن می‌تواند TSP را برای هر تعداد هسته فعال در پردازنده دلخواه تعیین کند. (که در ادامه به آن خواهیم پرداخت).

کدهای core و system به شکل زیر هستند:

```

1 class Core:
2     def __init__(self, type):
3         self.voltage_frequency_values = []
4         self.utilization = 0
5         self.type = type # high power ('HI') or low power ('LO')
6         self.scheduling = [] # 3-tuple including (task, start time, end time)
7
8     def find_first_free_time_slot_after(self, k, duration):
9         t = k
10        while True:
11            if all(not (start <= t < end or start < t + duration <= end) for _, start, end in self.scheduling):
12                return t
13            t += 1
14
15    def calculate_utilization(self):
16        total_time = sum(end - start for _, start, end in self.scheduling) # sum of active periods of the core
17        self.utilization = total_time
18
19    def schedule(self, task, start_time, duration):
20        self.scheduling.append((task, start_time, start_time + duration))
21        self.calculate_utilization()

```

شکل ۹: core.py

```

7 class System:
8     def __init__(self, DAG, k):
9         self.DAG = DAG
10        self.TSP_LO = {0: None, 1: 3, 2: 1} # TSP for all possible number of active cores in 'Low Power Island'
11        self.TSP_HI = {0: None, 1: 5, 2: 3} # TSP for all possible number of active cores in 'High Power Island'
12        self.core_pairs = [(Core('HI'), Core('LO')) for _ in range(k)]
13
14    def get_best_core_pair(self):
15        return min(self.core_pairs, key=lambda pair: pair[0].utilization + pair[1].utilization)
16
17    def get_max_active_cores_in_interval(self, t, duration, core_type):
18        active_count = defaultdict(int)
19
20        for hi_core, lo_core in self.core_pairs:
21            core = hi_core if core_type == 'HI' else lo_core
22
23            for _, start, end in core.scheduling:
24                overlap_start = max(t, start)
25                overlap_end = min(t + duration, end)
26
27                if overlap_start < overlap_end:
28                    for time_point in range(overlap_start, overlap_end):
29                        active_count[time_point] += 1
30
31        return max(active_count.values(), default=0)
32
33    def check_TSP(self, t, duration, peak_power, core_type):
34        active_cores = self.get_max_active_cores_in_interval(t, duration, core_type)
35        if core_type == "HI":
36            tsp_constraint = self.TSP_HI[active_cores]
37        else:
38            tsp_constraint = self.TSP_LO[active_cores]
39
40        if tsp_constraint == None or tsp_constraint >= peak_power:
41            return True
42        return False
43
44    def get_finish_times_for_task(self, task_id):
45        finish_times = []

```

شکل ۱۰: system.py


```

43
44 def get_finish_times_for_task(self, task_id):
45     finish_times = []
46
47     for hi_core, lo_core in self.core_pairs:
48         for _, s, e in hi_core.scheduling:
49             if _id == task_id:
50                 finish_times.append(e)
51         for _, s, e in lo_core.scheduling:
52             if _id == task_id:
53                 finish_times.append(e)
54
55     return finish_times
56
57 def export_system_full_to_json(self, filename="system_scheduled.json"):
58     system_data = {
59         "DAG": self.DAG.to_dict(),
60         "TSP_LO": self.TSP_LO,
61         "TSP_HI": self.TSP_HI,
62         "core_pairs": []
63     }
64
65     for i, (hi_core, lo_core) in enumerate(self.core_pairs):
66         core_pair_data = {
67             "pair_id": i + 1,
68             "HI_core": hi_core.core_to_dict(),
69             "LO_core": lo_core.core_to_dict()
70         }
71         system_data["core_pairs"].append(core_pair_data)
72
73     with open(filename, "w") as f:
74         json.dump(system_data, f, indent=4)
75
76     print("✅ System Scheduled saved in file 'system_scheduled.json'")
77
78
79
80
81

```

شکل ۱۱: system.py

نکته‌ای که می‌خواهم به آن اشاره کنم بخش زیر از کد system.py است:

```

1     self.TSP_LO = {0: None, 1: 3, 2: 1} # TSP for all possible
      number of active cores in 'Low Power Island'
2     self.TSP_HI = {0: None, 1: 5, 2: 3} # TSP for all possible
      number of active cores in 'High Power Island'

```

برای مثال در این مورد، ما دو جفت هسته داریم که هر جفت شامل یک هسته big یا HI و یک هسته LITTLE یا LO است. وقتی نوشته شده TSP_LO برای مقدار ۱ برابر ۳ است، یعنی در صورتی که در میان هسته‌های کم مصرف یا LITTLE، یک هسته فعال باشد، ما کسیمم توان مصرفی آن هسته باید ۳ وات باشد تا محدودیت TSP رعایت شود. یا برای مثال وقتی TSP_HI برای ۲ برابر ۳ است، یعنی اگر از میان دو هسته HI یا big، هر دو فعال باشند، توان مصرفی بیشینه آنها باید ۳ وات باشد.

تصاویر زیر نیز مربوط به الگوریتم TASS است که مشابه شبه‌کد مقاله زده شده است.

```

12 def TASS_algorithm(system, dag):
13     stack = []
14     remaining = set(dag.tasks)
15     all_children = {
16         task.id: set(child.id for child in task.children) for task in dag.tasks
17     }
18
19     # Find tasks that have no children (leaf tasks in the remaining DAG)
20     while remaining:
21         leaves = [t for t in remaining if not any(child in remaining for child in t.children)]
22         if not leaves:
23             break
24         leaf = max(leaves, key=lambda t: t.deadline)
25         stack.append(leaf)
26         remaining.remove(leaf)
27
28     while stack:
29         task = stack.pop()
30         C_hp, C_lp = system.get_best_core_pair()
31
32         # === Primary Scheduling ===
33         preds = task.parents
34
35         k = max((
36             max(system.get_finish_times_for_task(p.id), default=0)
37             for p in preds
38         ), default=0)
39
40         t = k
41         t_free = None
42         while t <= task.deadline - task.wcet_HI:
43             t_free = C_hp.find_first_free_time_slot_after(t, task.wcet_HI)
44             if system.check_TSP(t_free, task.wcet_HI, task.peak_power_value_HI, 'HI'):
45                 C_hp.schedule(task, t_free, task.wcet_HI)
46                 break
47             t += 1
48
49         if t > task.deadline - task.wcet_HI:
50             return False # UNSCHEDULABLE!

```

شکل ۱۲: TASS.py

```

49         if t > task.deadline - task.wcet_HI:
50             return False # UNSCHEDULABLE!
51
52         finish_primary = t_free + task.wcet_HI
53
54         # === Backup Scheduling ===
55         t = finish_primary
56         while True:
57             t_free = C_lp.find_first_free_time_slot_after(t, task.wcet_LO)
58             if system.check_TSP(t_free, task.wcet_LO, task.peak_power_value_LO, 'LO'):
59                 C_lp.schedule(task, t_free, task.wcet_LO)
60                 break
61             t += 1
62
63         return True # SCHEDULABLE!
64
65 def plot_schedule(system):
66     num_core_pairs = len(system.core_pairs)
67
68     # Separate and order cores: first all HI, then all LO
69     hi_cores = [(f"HP_C{i+1}", pair[0]) for i, pair in enumerate(system.core_pairs)]
70     lo_cores = [(f"LP_C{i+1}", pair[1]) for i, pair in enumerate(system.core_pairs)]
71     all_cores = hi_cores + lo_cores
72
73     # Reverse for Gantt chart so HI cores appear first top-down
74     gantt_cores = list(reversed(all_cores))
75
76     fig, (gantt_ax, *power_axes) = plt.subplots(1 + len(all_cores), 1,
77                                                figsize=(16, 3 + 2.5 * len(all_cores)),
78                                                gridspec_kw={'height_ratios': [4] + [2]*len(all_cores)})
79
80     # --- Top: Gantt chart ---
81     y_labels = []
82     yticks = []
83     yt = 0
84     max_time = 0
85
86     for label, core in gantt_cores:

```

شکل ۱۳: TASS.py

در نهایت با ورودی دادن جیسون DAG ساخته شده از طریق کد سازنده DAG به این کد زمان‌بند، زمان‌بندی انجام شده و به صورت گرافیکی نیز در یک نمودار ذخیره می‌شود.

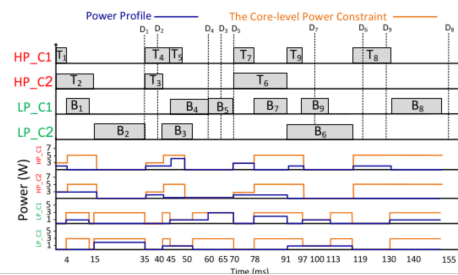
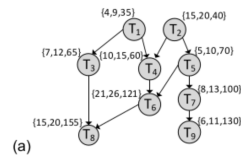
برای مثال من از یکی از مثال‌های خود مقاله شروع کردم:

Table 1. The peak power consumption of tasks running on different types of cores

High Performance Island		Low Power Island	
Task Name	Power value	Task Name	Power value
T ₁	2W	T ₁	1W
T ₂	3W	T ₂	2W
T ₃	2W	T ₃	1W
T ₄	2W	T ₄	1W
T ₅	4W	T ₅	3W
T ₆	2W	T ₆	1W
T ₇	3W	T ₇	2W
T ₈	2W	T ₈	1W
T ₉	2W	T ₉	1W

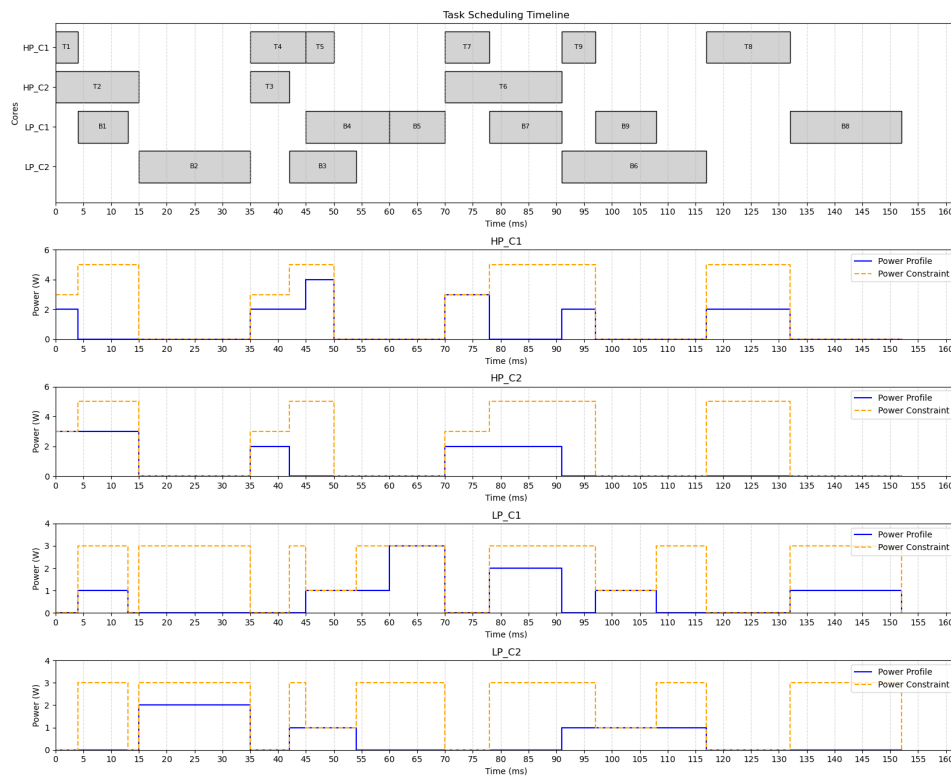
Table 2. The core-level power constraint information of the cores at different situations when the number of active cores are different on different islands

High Performance Island		Low Power Island	
Number of active cores	The core-level Power constraint	Number of active cores	The core-level Power constraint
1	5W	1	3W
2	3W	2	1W



شکل ۱۴: نمونه مقاله از TASS

در این نمونه، مقادیر peak power وظایف برای هر دو نوع هسته HI و LO مشخص شده، همچنین TSP ها نیز مشخص شده‌اند. همچنین در DAG وابستگی‌ها نیز مشخص شده‌اند و ۳ تایی نوشته شده، به ترتیب $(WCET_{LO}, WCET_{HI}, Deadline)$ هستند. همین ورودی را به صورت جیسون به زمان‌بند آفلاین می‌دهم و نتیجه را قرار می‌دهم، مشاهده می‌شود که زمان‌بند ما خروجی‌ای مشابه به خروجی مقاله می‌دهد.

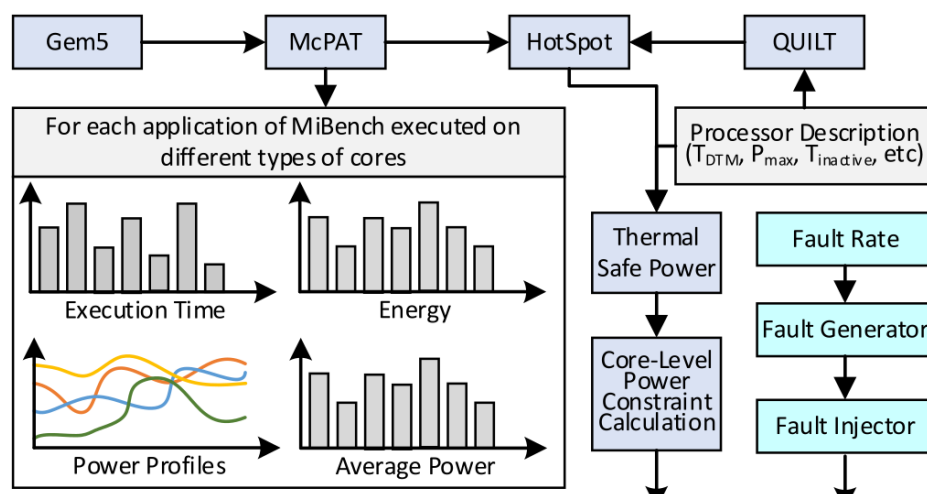


شکل ۱۵: خروجی TASS

در نهایت خروجی زمان‌بندی را در فایل جیسون ذخیره کرده و برای ادامه کار (زمان‌بندی بخش runtime) استفاده می‌کنیم.

۶.۳ کار کردن با شبیه‌سازها برای گرفتن مقادیر ورودی

حال نوبت به کار با شبیه‌سازها می‌رسد، به طور کلی شمای استفاده از شبیه‌سازها طبق مقاله اصلی به شکل زیر است:



شکل ۱۶: ترتیب استفاده از شبیه‌سازها

بنابراین در ابتدا باید با استفاده از شبیه‌ساز gem5، وظایف MiBench را روی دو نوع هسته ذکر شده در مقاله که ARM cortex A7 و ARM cortex A15 هستند اجرا کنیم و پروفایل بدترین زمان اجرا WCET را بگیریم.

Table 4. The details of system configuration

Parameter	Configuration	
	LITTLE island	big island
Core Type	ARM Cortex-A7	ARM Cortex-A15
Machine Type	In-Order	Out-Of-Order
Core Volt. And Freq.	[0.78V, 0.4GHz] to [0.93V, 1.4GHz]	[0.89V, 0.8GHz] to [0.9995V, 2GHz]
Feature Size	14 nm	
Core Microarchitecture	ARMv7-A	
L1 Cache	32KB, 8KB block-width, 4-way	
L2 Cache	2MB, 16-way	
Memory	2GB, 32-bit LPDDR3e	

شکل ۱۷: اطلاعات هسته‌های ARM cortex A7، ARM cortex A15

مطابق با جستجوهای انجام شده، متوجه شدم که هسته‌های A7 و A15 توسط خود gem5 در فایل‌هایی در مسیر configs/common/cores/arm/ex5_LITTLE.py و configs/common/cores/arm/ex5_big.py شبیه‌سازی شده است.

این موضوع را از کامنتی که سازندگان در این دو فایل کانفیگ قرار داده‌اند متوجه شدم.

```

27
28 from m5.objects import *
29
30 # -----
31 #           ex5 big core (based on the ARM Cortex-A15)
32 # -----
33
34

```

شکل ۱۸: برای مثال در فایل ex5_big.py اینگونه نوشته شده است.

سپس برای شبیه‌سازی یک سیستم کامل با استفاده از این هسته‌ها به طور مجزا، دو فایل کانفیگ را خودم نوشتم که در مسیر src/configs نام‌های se_ARM_cortex_a15.py و se_ARM_cortex_a7.py موجود هستند. برای مثال کانفیگ نوشته شده برای A15 را قرار می‌دهم.

```

1  from m5.objects import *
2  from m5.util import addToPath
3  import m5
4  import argparse
5
6  # کن import از فایل خودت
7
8  m5.util.addToPath("../..")
9
10 import devices
11 from common import (
12     FSConfig,
13     ObjectList,
14     Options,
15     SysPaths,
16 )
17 from common.cores.arm import (
18     ex5_big,
19     ex5_LITTLE,
20 )
21 from devices import (
22     AtomicCluster,
23     FastmodelCluster,
24     KvmCluster,
25 )
26 from common.cores.arm.ex5_big import ex5_big, L1I, L1D, L2 # باشه ex5_big.py فرض کن این فایل اسمش
27
28 parser = argparse.ArgumentParser()
29 parser.add_argument("binary", help="Path to ARM binary")
30 parser.add_argument("args", nargs=argparse.REMAINDER, help="Arguments to the binary")
31 args = parser.parse_args()
32
33 system = System()
34 system.clk_domain = SrcClockDomain(clock="1GHz", voltage_domain=VoltageDomain())
35 system.mem_mode = "timing"
36 system.mem_ranges = [AddrRange("2GiB")]
37
38 # CPU
39 system.cpu = ex5_big(cpu_id=0)
40 system.cpu.clk_domain = SrcClockDomain(clock="2.0GHz", voltage_domain=VoltageDomain(voltage="0.9995V"))

```

شکل ۱۹: se_ARM_cortex_a15.py

```

40 system.cpu.clk_domain = SrcClockDomain(clock="2.0GHz", voltage_domain=VoltageDomain(voltage="0.9995V"))
41
42 # کپیها
43 system.cpu.icache = L1I(cpu_side = system.cpu.icache_port)
44 system.cpu.dcache = L1D(cpu_side = system.cpu.dcache_port)
45
46 # اتصال L2
47 system.l2cache = L2()
48 system.tol2bus = L2XBar()
49 system.cpu.icache.mem_side = system.tol2bus.cpu_side_ports
50 system.cpu.dcache.mem_side = system.tol2bus.cpu_side_ports
51 system.l2cache.cpu_side = system.tol2bus.mem_side_ports
52
53 # حافظه
54 system.membus = SystemXBar()
55 system.l2cache.mem_side = system.membus.cpu_side_ports
56
57 system.system_port = system.membus.cpu_side_ports
58
59 system.mem_ctrl = MemCtrl()
60 system.mem_ctrl.dram = LPDDR3_1600_1x32()
61 system.mem_ctrl.dram.range = system.mem_ranges[0]
62 system.mem_ctrl.port = system.membus.mem_side_ports
63
64 # بارگذاری باینری
65 system.workload = SEWorkload.init_compatible(args.binary)
66 system.cpu.createInterruptController()
67
68 process = Process()
69 process.cmd = [args.binary] + args.args
70 system.cpu.workload = process
71 system.cpu.createThreads()
72
73 root = Root(full_system=False, system=system)
74 m5.instantiate()
75
76 print("Starting simulation with ex5_big core (Cortex-A15-like)")
77 exit_event = m5.simulate()
78 print(f"Exiting @ tick {m5.curTick()} because {exit_event.getCause()}")
79

```

شکل ۲۰: se_ARM_cortex_a15.py

سپس باینری‌های ARM مربوط به وظایف بنچمارک MiBench را از طریق این لینک دانلود کرده و برخی از آنها

را روی این هسته‌ها اجرا می‌کنیم.

دستور اجرا برای مثال به این شکل است:

```
1 ../../../../build/ARM/gem5.opt ../../../../configs/example/arm/  
se_ARM_cortex_a7.py "bitcnts 75000"
```

علت نقطه‌ها نیز صدا زدن برنامه از فولدری داخل tests است.

بنابراین خروجی اجرای gem5 را برای برخی وظایف ذخیره کردیم که در فولدر src/results موجود است.

```
1  
2 ----- Begin Simulation Statistics -----  
3 simSeconds          0.016965      # Number of seconds simulated (Second)  
4 simTicks            16964964500    # Number of ticks simulated (Tick)  
5 finalTick           16964964500    # Number of ticks from beginning of simulation (r  
6 simFreq             1000000000000   # The number of ticks per simulated second ((Tic  
7 hostSeconds         63.36          # Real time elapsed on the host (Second)  
8 hostTickRate        267735937      # The number of ticks simulated per host second (  
9 hostMemory          2288312        # Number of bytes of host memory used (Byte)  
10 simInsts            43590752       # Number of instructions simulated (Count)  
11 simOps              44497625       # Number of ops (including micro ops) simulated (  
12 hostInstRate        687935        # Simulator instruction rate (inst/s) ((Count/Sec  
13 hostOpRate          702247        # Simulator op (including micro ops) rate (op/s)  
14 system.clk_domain.clock 1000     # Clock period in ticks (Tick)  
15 system.clk_domain.voltage_domain.voltage 1      # Voltage in Volts (Volt)  
16 system.cpu.numCycles 33929930      # Number of cpu cycles simulated (Cycle)  
17 system.cpu.cpi       0.778375      # CPI: cycles per instruction (core level) ((Cycl  
18 system.cpu.ipc       1.284729      # IPC: instructions per cycle (core level) ((Coun  
19 system.cpu.numWorkItemsStarted 0      # Number of work items this cpu started (Count)  
20 system.cpu.numWorkItemsCompleted 0      # Number of work items this cpu completed (Count)  
21 system.cpu.instsAdded 45250391     # Number of instructions added to the IQ (exclud  
22 system.cpu.nonSpecInstsAdded 169     # Number of non-speculative instructions added to  
23 system.cpu.instsIssued 45101856     # Number of instructions issued (Count)  
24 system.cpu.squashedInstsIssued 73900 # Number of squashed instructions issued (Count)  
25 system.cpu.squashedInstsExamined 752934 # Number of squashed instructions iterated over d  
26 system.cpu.squashedOperandsExamined 676430 # Number of squashed operands that are examined a  
27 system.cpu.squashedNonSpecRemoved 34    # Number of squashed non-spec instructions that w  
28 system.cpu.numIssuedDist::samples 33836412 # Number of insts issued each cycle (Count)  
29 system.cpu.numIssuedDist::mean 1.332938 # Number of insts issued each cycle (Count)  
30 system.cpu.numIssuedDist::stdev 0.721514 # Number of insts issued each cycle (Count)  
31 system.cpu.numIssuedDist::underflows 0      # Number of insts issued each cycle (Count)  
32 system.cpu.numIssuedDist::0 4070453 12.03% 0.00% # Number of insts issued each cycle (Count)  
33 system.cpu.numIssuedDist::1 15409337 45.54% 57.57% # Number of insts issued each cycle (Count)  
34 system.cpu.numIssuedDist::2 13377556 39.54% 97.11% # Number of insts issued each cycle (Count)  
35 system.cpu.numIssuedDist::3 978857 2.89% 100.00% # Number of insts issued each cycle (Count)  
36 system.cpu.numIssuedDist::4 209 0.00% 100.00% # Number of insts issued each cycle (Count)
```

شکل ۲۱: خروجی نمونه gem5

مقادیر simSeconds به عنوان مقدار WCET وظیفه در نظر گرفته می‌شود.

۲.۶.۳ McPat

حال نوبت کار با شبیه‌ساز McPat است. در ابتدا نیاز است که فایل xml ورودی برای این شبیه‌ساز را تولید کنیم. تولید این فایل به طور دستی بسیار سخت است. بنابراین یک کد پایتون نوشتم که با ورودی گرفتن فایل‌های config.json، stats.txt، فایل xml مربوطه را تولید می‌کند. از آنجا که این کد، جز پیاده‌سازی، مفهوم دیگری ندارد، از توضیح و قرار دادن عکس کد آن صرف نظر می‌کنم. کد در مسیر src/converters/gem5_to_mcpat موجود است.

حال تمامی نتایجی که با gem5 بدست آمده را از طریق این برنامه مبدل، به فایل xml تبدیل کرده و به McPat ورودی می‌دهم. مجدد خروجی McPat را به فولدرهای مربوطه در src/result اضافه می‌کنم. برای مثال یک نمونه از آن را ببینیم:

```

1 McPAT (version 1.3 of Feb, 2015) is computing the target processor...
2
3
4 McPAT (version 1.3 of Feb, 2015) results (current print level is 5)
5 *****
6 Technology 28 nm
7 Using Long Channel Devices When Appropriate
8 Interconnect metal projection= conservative interconnect technology projection
9 Core clock Rate(MHz) 1400
10 *****
11
12 Processor:
13 Area = 11.6836 mm^2
14 Peak Power = 2.67584 W
15 Total Leakage = 0.293864 W
16 Peak Dynamic = 2.38198 W
17 Subthreshold Leakage = 0.245436 W
18 Gate Leakage = 0.0484285 W
19 Runtime Dynamic = 0.202006 W
20
21 Total Cores: 1 cores
22 Device Type= ITRS low operating power device type
23 Area = 4.8796 mm^2
24 Peak Dynamic = 1.49959 W
25 Subthreshold Leakage = 0.0932375 W
26 Gate Leakage = 0.00229782 W
27 Runtime Dynamic = 0.185082 W
28
29 Total L2s:
30 Device Type= ITRS high performance device type
31 Area = 3.10364 mm^2
32 Peak Dynamic = 0.729031 W
33 Subthreshold Leakage = 0.066748 W
34 Gate Leakage = 0.00569198 W
35 Runtime Dynamic = 5.78457e-06 W
36

```

شکل ۲۲: خروجی نمونه McPat

۳.۶.۳ HotSpot

مجدداً برای تبدیل خروجی حاصل از McPat به ورودی HotSpot و استفاده از آن برای شبیه‌سازی با HotSpot، یک برنامه پایتون نوشتیم که با ورودی گرفتن خروجی‌های McPat و Gem5، ورودی مربوطه برای HotSpot را تولید می‌کند. مجدداً از توضیح دادن کد این برنامه پرهیز می‌کنم. این کد در مسیر `src/converters/mcpat_to_hotspot` قرار دارد.

پس از آن فایل‌های `floorplan` و `config` مربوط به `hotspot` و `ptrace` تولید می‌شوند که برای نمونه، فایل `floor-plan` یکی را قرار می‌دهم.

```

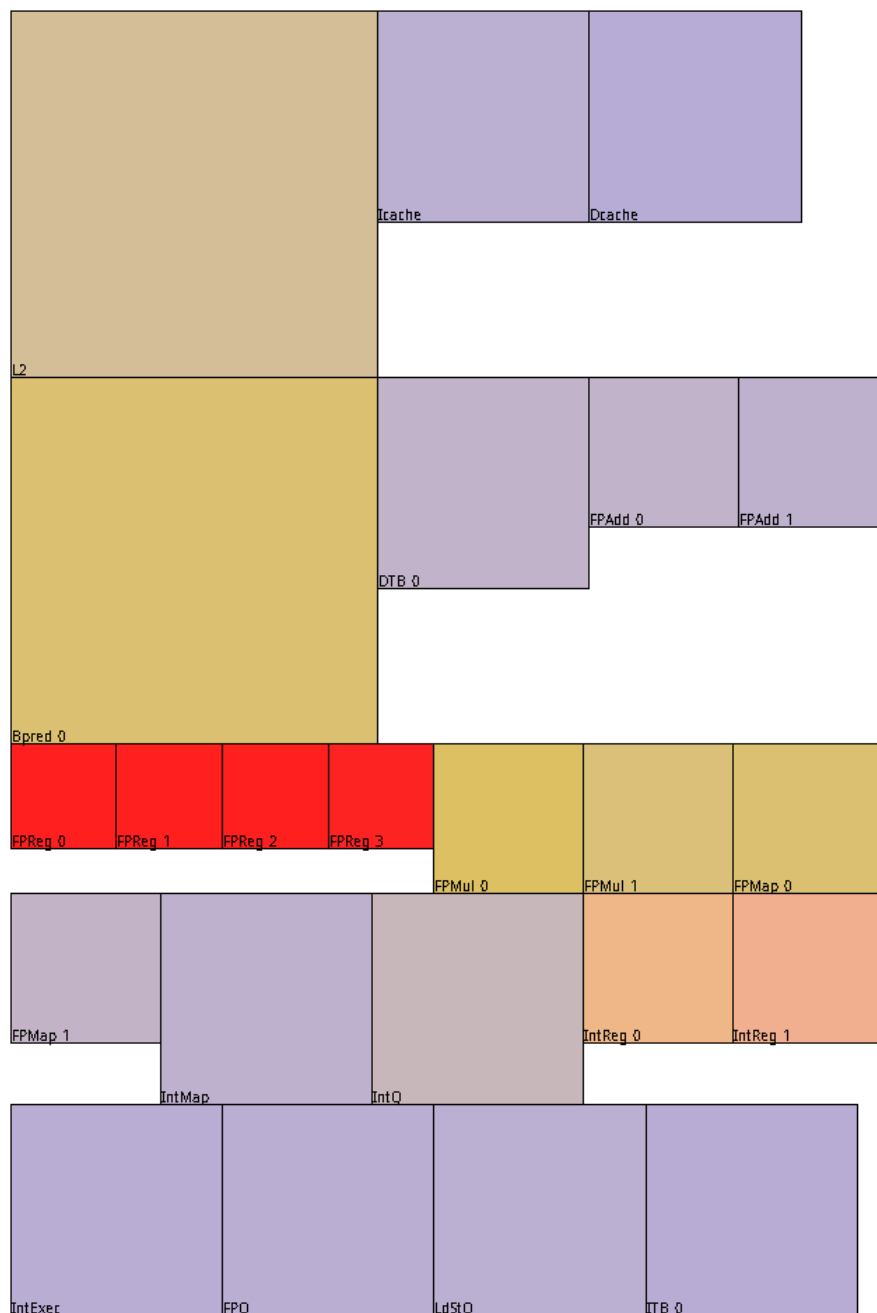
floorplan.flp x
src > codes > converters > mcpat_to_hotspot > floorplan.flp
1 #name width height x y
2 L2 0.001732 0.001732 0.000000 0.000000
3 Icache 0.001000 0.001000 0.001732 0.000000
4 Dcache 0.001000 0.001000 0.002732 0.000000
5 Bpred_0 0.001732 0.001732 0.000000 0.001732
6 DTB_0 0.001000 0.001000 0.001732 0.001732
7 FPAdd_0 0.000707 0.000707 0.002732 0.001732
8 FPAdd_1 0.000707 0.000707 0.003439 0.001732
9 FPReg_0 0.000500 0.000500 0.000000 0.003464
10 FPReg_1 0.000500 0.000500 0.000500 0.003464
11 FPReg_2 0.000500 0.000500 0.001000 0.003464
12 FPReg_3 0.000500 0.000500 0.001500 0.003464
13 FPMul_0 0.000707 0.000707 0.002000 0.003464
14 FPMul_1 0.000707 0.000707 0.002707 0.003464
15 FPMMap_0 0.000707 0.000707 0.003414 0.003464
16 FPMMap_1 0.000707 0.000707 0.000000 0.004171
17 IntMap 0.001000 0.001000 0.000707 0.004171
18 IntQ 0.001000 0.001000 0.001707 0.004171
19 IntReg_0 0.000707 0.000707 0.002707 0.004171
20 IntReg_1 0.000707 0.000707 0.003414 0.004171
21 IntExec 0.001000 0.001000 0.000000 0.005171
22 FPQ 0.001000 0.001000 0.001000 0.005171
23 LdStQ 0.001000 0.001000 0.002000 0.005171
24 ITB_0 0.001000 0.001000 0.003000 0.005171
25

```

شکل ۲۳: فایل نمونه floorplan

QUILT ۴.۶.۳

شبیه‌ساز QUILT را صرفاً برای مشاهده floorplan و همچنین power trace حاصل از HotSpot استفاده کردم و نتیجه به شکل زیر بود:



شکل ۲۴: نتیجه نمایش floorplan و ptrace با استفاده از QUILT، توجه شود که این پردازنده صرفاً نمادین است و قرارگیری‌ها غیرواقعی هستند.

۵.۶.۳ TSP

و در نهایت شبیه‌ساز TSP برای بدست آوردن TSP ها بکار می‌رود. کافی است خروجی‌های HotSpot را به شکلی درست به TSP ورودی دهیم. از آنجا که انجام این کار زمان‌بر بود و نیازی به مقادیر دقیق هم در این موضوع نداشتیم، صرفاً یک نمونه از خود TSP را اجرا گرفتیم که نحوه عملکرد آن مشاهده شود:

```
TSP per-core(25) [Watts]: 6.93872
TSP per-core(26) [Watts]: 6.75557
TSP per-core(27) [Watts]: 6.58499
TSP per-core(28) [Watts]: 6.42372
TSP per-core(29) [Watts]: 6.27036
TSP per-core(30) [Watts]: 6.12529
TSP per-core(31) [Watts]: 5.98799
TSP per-core(32) [Watts]: 5.85705
TSP per-core(33) [Watts]: 5.73277
TSP per-core(34) [Watts]: 5.61275
TSP per-core(35) [Watts]: 5.49818
TSP per-core(36) [Watts]: 5.38937
TSP per-core(37) [Watts]: 5.28503
TSP per-core(38) [Watts]: 5.18545
TSP per-core(39) [Watts]: 5.09001
TSP per-core(40) [Watts]: 4.9989
TSP per-core(41) [Watts]: 4.91124
TSP per-core(42) [Watts]: 4.82669
TSP per-core(43) [Watts]: 4.74559
TSP per-core(44) [Watts]: 4.66735
TSP per-core(45) [Watts]: 4.59195
TSP per-core(46) [Watts]: 4.52086
TSP per-core(47) [Watts]: 4.45208
TSP per-core(48) [Watts]: 4.38571
TSP per-core(49) [Watts]: 4.32166
TSP per-core(50) [Watts]: 4.25953
TSP per-core(51) [Watts]: 4.19929
TSP per-core(52) [Watts]: 4.14097
TSP per-core(53) [Watts]: 4.0844
TSP per-core(54) [Watts]: 4.02971
TSP per-core(55) [Watts]: 3.97665
TSP per-core(56) [Watts]: 3.92531
TSP per-core(57) [Watts]: 3.8754
TSP per-core(58) [Watts]: 3.82697
TSP per-core(59) [Watts]: 3.78011
TSP per-core(60) [Watts]: 3.73473
TSP per-core(61) [Watts]: 3.69051
TSP per-core(62) [Watts]: 3.64767
TSP per-core(63) [Watts]: 3.60612
TSP per-core(64) [Watts]: 3.5663
(base) farzam@farzam-ASUS-TUF-Gaming-F15-FX507ZE-FX507ZE:~/Farzam/Sharif/Term6/Low_
.1$
```

شکل ۲۵: استفاده از شبیه‌ساز TSP برای بدست آوردن مقادیر

این شبیه‌سازی برای یک پردازنده ۶۴ هسته‌ای انجام شده که مقدار درون پرانتزها، تعداد هسته‌های فعال از بین این ۶۴ هسته را نشان داده و مقدار توان گفته شده، همان TSP است.

به عنوان حرکت نهایی، صرفاً مقادیر نهایی peak power و WCET را برای دو هسته A7, A15 و برای برخی از وظایف MiBench به شرح زیر است:

	A	B	C	D	E
1	Mibench Task	Large/Small	Core	Execution Time (s)	Peak Power (W)
2	Basicmath	Small	ARM cortex a7	0.286784	2.67778
3		Large	ARM cortex a7	-	-
4		Small	ARM cortex a15	0.183907	3.37988
5		Large	ARM cortex a15	-	-
6	Bitcount	Small	ARM cortex a7	0.032797	2.67584
7		Large	ARM cortex a7	-	-
8		Small	ARM cortex a15	0.016965	3.3784
9		Large	ARM cortex a15	-	-
10	CRC32	Small	ARM cortex a7	0.131082	2.67584
11		Large	ARM cortex a7	-	-
12		Small	ARM cortex a15	0.105481	3.3784
13		Large	ARM cortex a15	-	-
14	Dijkstra	Small	ARM cortex a7	0.039537	2.67633
15		Large	ARM cortex a7	-	-
16		Small	ARM cortex a15	0.025674	3.37938
17		Large	ARM cortex a15	-	-
18	FFT	Small	ARM cortex a7	0.258702	2.67584
19		Large	ARM cortex a7	-	-
20		Small	ARM cortex a15	0.156178	3.3784
21		Large	ARM cortex a15	-	-
22	Jpeg	Small	ARM cortex a7	-	-
23		Large	ARM cortex a7	-	-
24		Small	ARM cortex a15	-	-
25		Large	ARM cortex a15	-	-
26	Patricia	Small	ARM cortex a7	0.161125	2.68215
27		Large	ARM cortex a7	-	-
28		Small	ARM cortex a15	0.099492	3.38383
29		Large	ARM cortex a15	-	-
30	Qsort	Small	ARM cortex a7	0.023569	2.6773
31		Large	ARM cortex a7	-	-
32		Small	ARM cortex a15	0.01063	3.38136
33		Large	ARM cortex a15	-	-

شکل ۲۶: profiling انجام شده روی وظایف MiBench

فایل برخی از نتایج شبیه‌سازی نیز در فولدر src/results موجود است.

۷.۳ زمان‌بندی در runtime

زمان‌بندی در Runtime اینگونه انجام می‌شود که برای وظایف اصلی، نرخ خطا در نظر می‌گیریم، اما از آنجا که وقوع نرخ خطا در وظایف پشتیبان، به معنی نابودی سیستم است، از در نظر گرفتن آن صرف نظر می‌کنیم (در مقاله هم اشاره‌ای به آن نشده).

بنابراین قابلیت اطمینان وظیفه را طبق رابطه زیر محاسبه می‌کنیم:

Created by [10] [17].

$$\lambda(V_i) = \lambda_0 \times 10^{\frac{V_{max}-V_i}{d}} \quad (4)$$

Where λ_0 is the transient fault rate at V_{max} , and d determines the sensitivity of the system to voltage scaling. The functional reliability of a task can be written as [2][7][9][38]:

$$R(T_i) = e^{-\lambda(V_i) \times wc_i} \quad (5)$$

شکل ۲۷: رابطه قابلیت اطمینان

حال کد شبیه‌سازی در runtime را در ادامه قرار می‌دهم:

```

1  import math
2  import random
3
4  from codes.runtime_simulator.input.DFVS_levels import *
5  from codes.offline_simulator.scheduler.TASS import plot_schedule
6
7  LAMBDA0 = 1e-6
8  D = 0.15      # 28nm tech
9  V_MAX = max(HI_core_level[i][0] for i in range(len(HI_core_level)))
10
11 def lambda_vi(voltage):
12     return LAMBDA0 * (10 ** ((V_MAX - voltage) / D))
13
14
15 def reliability(task_time, voltage):
16     lam = lambda_vi(voltage)
17     return math.exp(-lam * task_time)
18 #return 0.5
19
20
21 def runtime_scheduling(system_restored):
22     max_time = 0
23     for hi_core, lo_core in system_restored.core_pairs:
24         for _, _, end in hi_core.scheduling:
25             max_time = max(max_time, end)
26         for _, _, end in lo_core.scheduling:
27             max_time = max(max_time, end)
28
29     finished_tasks = set()
30     failed_tasks = set()
31
32     for current_time in range(max_time + 1):
33         for hi_core, lo_core in system_restored.core_pairs:
34             # HI Core
35             for task, start, end in hi_core.scheduling:
36                 if start == current_time:

```

شکل ۲۸: زمان‌بند در runtime

```

21 def runtime_scheduling(system_restored):
22     for hi_core, lo_core in system_restored.core_pairs:
23         for _, _, end in hi_core.scheduling:
24             max_time = max(max_time, end)
25         for _, _, end in lo_core.scheduling:
26             max_time = max(max_time, end)
27
28     finished_tasks = set()
29     failed_tasks = set()
30
31     for current_time in range(max_time + 1):
32         for hi_core, lo_core in system_restored.core_pairs:
33             # HI Core
34             for task, start, end in hi_core.scheduling:
35                 if start == current_time:
36                     duration = (end - start) # converting ms to s
37
38                     voltage = V_MAX
39
40                     r = reliability(duration, voltage)
41                     success = random.random() < r
42
43
44                     if success:
45                         finished_tasks.add(task.id)
46                     else:
47                         failed_tasks.add(task.id)
48                         print(f"✗ Task {task.id} FAILED on HI Core")
49
50             # L0 Core
51             for task, start, end in lo_core.scheduling:
52                 if task.id in finished_tasks:
53                     lo_core.scheduling.remove((task, start, end))
54
55     plot_schedule(system_restored)
56
57

```

شکل ۲۹: زمان‌بند در runtime

زمانی که وظیفه‌ای اصلی به درستی اجرا شود، دیگر نیازی به اجرای وظیفه پشتیبان نیست و در اصل می‌توان وظیفه

پشتیان آن را اجرا نکرد و از تکنیک DPM یا Dynamic Power Management روی هسته پشتیبان استفاده کرد تا توان مصرفی کمتری مصرف شود.

از آنجا که در هسته اصلی، از ولتاژ ماکسیمم استفاده می‌شود، طبق روابط خواهیم داشت:

$$\lambda_0 = 10^{-6}$$

$$V_{max} = V$$

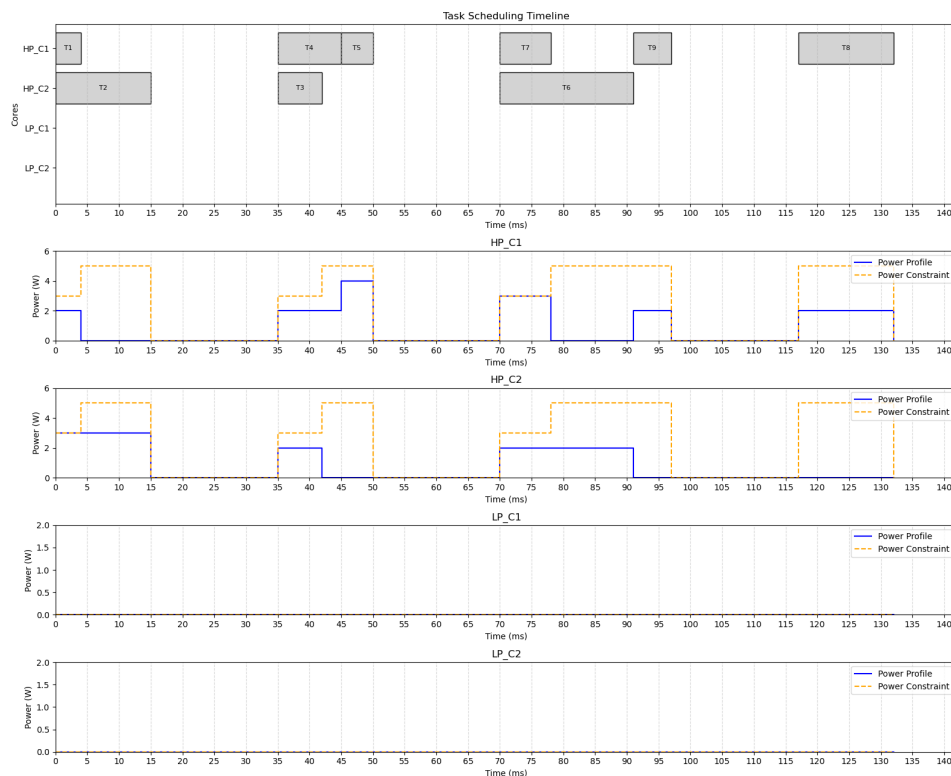
$$d = 0.15 \quad \text{for } 28 \text{ nm technology}$$

$$\lambda(V) = \lambda_0 \times 10^{\frac{V_{max}-V}{d}} = \lambda_0$$

$$R(T) = e^{-\lambda(V) \times wcet}$$

و مقدار قابلیت اطمینان وظیفه اصلی طبق روابط بالا محاسبه شده، اگر اجرا به درستی انجام شود، وظیفه پشتیبان حذف شده و اجرا نمی‌شود و DPM روی هسته پشتیبان یا LO انجام می‌شود. در غیر اینصورت باید وظیفه پشتیبان اجرا شود.

از آنجا که مقدار Reliability وظیفه اصلی به قدری بالاست که تقریباً همواره درست انجام می‌شود، عموماً وظیفه‌های پشتیبان حذف می‌شوند. این موضوع در اجرای زیر قابل مشاهده هست (این اجرا در اصل ادامه مثال بخش offline است).



شکل ۳۰: نمونه زمان‌بندی runtime

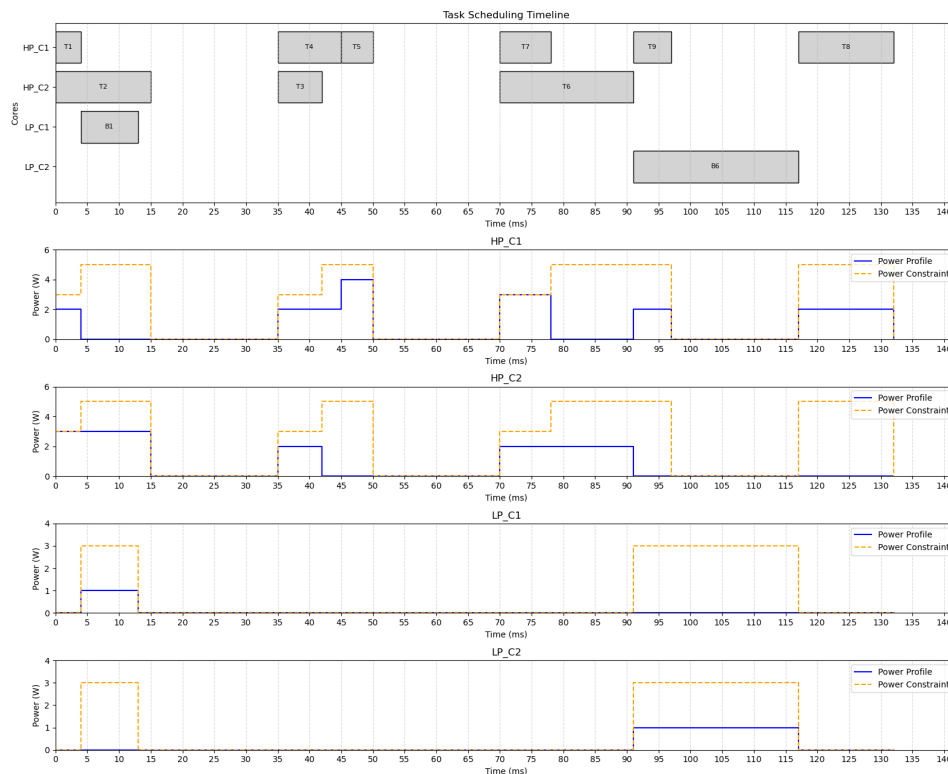
حال برای اطمینان از صحت زمان‌بند runtime، صرفاً مقدار Reliability را بجای رابطه قبلی، مقدار 0.75 ثابت قرار می‌دهم تا مشاهده شود در صورت خطا خوردن وظایف اصلی، پشتیبان‌ها اجرا می‌شوند.

```

● (base) farzam@farzam-ASUS-TUF-Gami
main.py
✗ Task 1 FAILED on HI Core
✗ Task 6 FAILED on HI Core

```

شکل ۳۱: دو وظیفه اصلی اول و ششم خطا خوردند



شکل ۳۲: بنابراین وظایف پشتیبان دو وظیفه اول و ششم اجرا می‌شوند.

نتیجه گیری

بنابر الگوریتم بخش آفلاین، ما توانستیم با در نظر گرفتن محدودیت توان مصرفی ایمن TSP برای هر هسته، محدودیت دمایی مدنظرمان را رعایت کنیم. همچنین از آنجا که احتمال خطا خوردن وظایف اصلی بسیار پایین است، اکثر وظایف اصلی روی هسته‌های big به درستی اجرا شده و می‌توانیم وظیفه پشتیبان را روی هسته LITTLE اجرا نکرده و روی هسته LITTLE در آن بازه‌های زمانی، تکنیک DPM بزنی که خود این کار، توان مصرفی کل و توان مصرفی بیشینه یا peak power را کاهش می‌دهد.